In [1]:
```
# ==============================================================================
# CELL 0: Project Path Setup
# ==============================================================================
import sys
from pathlib import Path

proj_root = Path.cwd()
src = proj_root / "src"
if str(src) not in sys.path:
    sys.path.append(str(src))

print("Project root:", proj_root)
print("Using src path:", src)
```

```
Project root: c:\Users\naikt\OneDrive\Desktop\tanmay_dissertation
Using src path: c:\Users\naikt\OneDrive\Desktop\tanmay_dissertation\src
```

In [2]:
```
# ==============================================================================
# CELL 1: Phase Header & Configuration
# ==============================================================================
from common.seed_logging import set_global_seed, get_logger
from common.reporting import Report
from common.paths import PathResolver

set_global_seed(42)
log = get_logger("phase02")
R = Report(phase=2)
P = PathResolver()
R.stamp("Phase 02 start")
```

In [3]:
```
# ==============================================================================
# CELL 2: Imports & Helper Functions
# ==============================================================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter
import re
import json
```

```python
# --- Column name normalization ---
def norm_cols(df: pd.DataFrame) -> pd.DataFrame:
    """Normalize column names to lowercase + stripped."""
    df = df.copy()
    df.columns = [str(c).strip().lower() for c in df.columns]
    return df

# --- Column finder with fallbacks ---
def col(df, candidates, default=None):
    """Return the first matching column (Series) by candidate names, else default."""
    for c in candidates:
        if c in df.columns:
            return df[c]
    return default

# --- Text length helper ---
def text_len(s: pd.Series) -> pd.Series:
    s = s.fillna("").astype(str)
    return s.str.len()

# --- Value counts to DataFrame ---
def value_counts_df(s: pd.Series, top=50, name="value", count_name="count"):
    vc = s.value_counts(dropna=False).head(top)
    return vc.rename(count_name).rename_axis(name).reset_index()

# --- Tokenization ---
TOKEN_SPLIT = re.compile(r"[\W_]+", re.UNICODE)

def tokenize_series(s: pd.Series):
    s = s.fillna("").astype(str).str.lower()
    return s.apply(lambda x: [t for t in TOKEN_SPLIT.split(x) if t])

def top_tokens(series_of_text, top=50):
    tokens = []
    for toks in tokenize_series(series_of_text):
        tokens.extend(toks)
    cnt = Counter(tokens)
    return pd.DataFrame(cnt.most_common(top), columns=["token", "count"])

# --- Skills parsing ---
```

```python
def parse_skills_column(s: pd.Series):
    """Parse a skills column that may contain semi-structured lists."""
    def split_one(x):
        if pd.isna(x):
            return []
        x = str(x)
        parts = re.split(r"[;,|/]+", x)
        return [p.strip().lower() for p in parts if p.strip()]
    return s.apply(split_one)

# --- Co-occurrence matrix ---
def cooccurrence_matrix(lists, top=30):
    """Build co-occurrence matrix for the top N items."""
    freq = Counter([item for sub in lists for item in sub])
    top_items = [it for it, _ in freq.most_common(top)]
    idx = {it: i for i, it in enumerate(top_items)}
    M = np.zeros((len(top_items), len(top_items)), dtype=int)

    for items in lists:
        uniq = sorted(set([it for it in items if it in idx]))
        for i in range(len(uniq)):
            for j in range(i, len(uniq)):
                a, b = idx[uniq[i]], idx[uniq[j]]
                M[a, b] += 1
                if a != b:
                    M[b, a] += 1
    return top_items, M

print("[OK] Helper functions defined")
```

```
[OK] Helper functions defined
```

In [4]:

```python
# ============================================================================
# CELL 3: Robust CSV Loader with Encoding Detection
# ============================================================================
def _detect_bom_encoding(path: Path):
    """Detect BOM encoding markers."""
    with open(path, "rb") as f:
        head = f.read(4)
    if head.startswith(b"\xff\xfe"):
        return "utf-16le"
```

```
    if head.startswith(b"\xfe\xff"):
        return "utf-16be"
    if head.startswith(b"\xef\xbb\xbf"):
        return "utf-8-sig"
    return None

def read_csv_flex(path: Path):
    """Robust CSV reader that tries multiple encodings and separators."""
    enc_first = _detect_bom_encoding(path)
    trials = []

    if enc_first:
        trials.append((enc_first, None))

    for enc in ["utf-16", "utf-8-sig", "cp1252", "latin1", "utf-8"]:
        for sep in [None, ",", "\t", ";", "|"]:
            pair = (enc, sep)
            if pair not in trials:
                trials.append(pair)

    last_err = None
    for enc, sep in trials:
        try:
            df = pd.read_csv(path, encoding=enc, sep=sep,
                             engine="python", on_bad_lines="skip")
            log.info(f"[read OK] {path.name} | encoding={enc} | sep={repr(sep)} | shape={df.shape}")
            return df
        except Exception as e:
            last_err = e
            continue

    raise RuntimeError(f"Could not read {path}. Last error: {last_err}")

print("[OK] Robust CSV loader defined")
```

```
[OK] Robust CSV loader defined
```

In [5]:

```
# ============================================================================
# CELL 4: Load Datasets
# ============================================================================
ph1 = P.phase_dir(1)
```

```
tab1 = ph1["tab"]

# Prefer canonical deliverable; fallback to dedup
jobs_path = tab1 / "jobs_it.csv"
if not jobs_path.exists():
    jobs_path = tab1 / "jobs_it_dedup.csv"
    log.info("Using fallback jobs_it_dedup.csv")

resumes_path = tab1 / "resumes_clean.csv"
ontology_path = P.dataset_path("ontology")

print("Reading:")
print(" ", jobs_path)
print(" ", resumes_path)
print(" ", ontology_path)

# Load files
jobs = pd.read_csv(jobs_path)
resumes = pd.read_csv(resumes_path)
ontology = read_csv_flex(ontology_path)

# Normalize headers
jobs = norm_cols(jobs)
resumes = norm_cols(resumes)
ontology = norm_cols(ontology)

print(f"\nShapes: jobs={jobs.shape}, resumes={resumes.shape}, ontology={ontology.shape}")
R.stamp(f"Loaded for EDA: jobs={jobs.shape}, resumes={resumes.shape}, ontology={ontology.shape}")
```

```
2025-10-11 01:22:30,275 | INFO | phase02 | [read OK] IT_Job_Roles_Skills.csv | encoding=cp1252 | sep=None | shape=(493, 4)
Reading:
  C:\Users\naikt\OneDrive\Desktop\tanmay_dissertation\outputs\phase01\tables\jobs_it.csv
  C:\Users\naikt\OneDrive\Desktop\tanmay_dissertation\outputs\phase01\tables\resumes_clean.csv
  C:\Users\naikt\OneDrive\Desktop\tanmay_dissertation\data\raw\IT_Job_Roles_Skills.csv

Shapes: jobs=(600, 5), resumes=(1200, 15), ontology=(493, 4)
```

In [6]:

```
# ============================================================================
# CELL 5: Quick Profiling & Schema Snapshots
# ============================================================================
def quick_profile(df: pd.DataFrame, name: str) -> pd.DataFrame:
```

```
    """Generate quick profile with nulls and uniques."""
    out = pd.DataFrame({
        "column": df.columns,
        "dtype": [str(t) for t in df.dtypes],
        "null_count": [df[c].isna().sum() for c in df.columns],
        "nunique": [df[c].nunique(dropna=True) for c in df.columns]
    })
    out.insert(0, "dataset", name)
    return out

prof_jobs = quick_profile(jobs, "jobs_p2")
prof_res = quick_profile(resumes, "resumes_p2")
prof_ont = quick_profile(ontology, "ontology_p2")

R.save_df(prof_jobs, "p2_jobs_profile.csv", index=False)
R.save_df(prof_res, "p2_resumes_profile.csv", index=False)
R.save_df(prof_ont, "p2_ontology_profile.csv", index=False)

print("[OK] Saved profiling tables")
print("\nSample - Jobs Profile:")
print(prof_jobs.head())
```

```
[OK] Saved profiling tables

Sample - Jobs Profile:
   dataset           column   dtype  null_count  nunique
0  jobs_p2           job_id   int64           0      600
1  jobs_p2         category  object           0        5
2  jobs_p2        job_title  object           0      385
3  jobs_p2  job_description  object           0      600
4  jobs_p2    job_skill_set  object           0      600
```

In [7]:

```
# ============================================================================
# CELL 6: Jobs - Basic Text Diagnostics
# ============================================================================
TITLE_CANDS = ["job_title", "title", "role", "position", "jobtitle"]
DESC_CANDS = ["job_description", "description", "desc", "jd", "details", "responsibilities"]
CAT_CANDS = ["category", "job_category", "department", "function"]

title_s = col(jobs, TITLE_CANDS, jobs.iloc[:, 0].astype(str))
desc_s = col(jobs, DESC_CANDS, pd.Series([""] * len(jobs), index=jobs.index))
```

```
cat_s = col(jobs, CAT_CANDS, pd.Series([""] * len(jobs), index=jobs.index))

# Save category counts
R.save_df(value_counts_df(cat_s, top=50, name="category"),
          "jobs_category_counts.csv", index=False)

# Title length histogram
fig = plt.figure()
ax = fig.add_subplot(111)
ax.hist(title_s.fillna("").astype(str).str.len(), bins=40)
ax.set_title("Job Title Length Distribution")
ax.set_xlabel("Characters")
ax.set_ylabel("Count")
R.save_fig(fig, "jobs_title_len_hist.png")
plt.close()

# Description length histogram (clipped at P99)
desc_len = desc_s.fillna("").astype(str).str.len()
fig = plt.figure()
ax = fig.add_subplot(111)
ax.hist(np.clip(desc_len, 0, np.percentile(desc_len, 99)), bins=50)
ax.set_title("Job Description Length Distribution")
ax.set_xlabel("Characters (clipped at P99)")
ax.set_ylabel("Count")
R.save_fig(fig, "jobs_desc_len_hist.png")
plt.close()

# Top tokens in title + desc
top_tok = top_tokens(title_s.astype(str) + " " + desc_s.astype(str), top=75)
R.save_df(top_tok, "jobs_top_tokens.csv", index=False)

print("[OK] Saved jobs diagnostics: category counts, length histograms, top tokens")
```

```
[OK] Saved jobs diagnostics: category counts, length histograms, top tokens
```

In [8]:

```
# ============================================================================
# CELL 7: Jobs - Skills Frequency & Co-occurrence
# ============================================================================
SKILL_CANDS = ["job_skill_set", "skills", "skillset", "required_skills", "top_skills"]
skills_col = col(jobs, SKILL_CANDS)
```

```
if skills_col is not None:
    job_skills_lists = parse_skills_column(skills_col)

    # Frequency
    freq = Counter([s for row in job_skills_lists for s in row])
    top_freq = pd.DataFrame(freq.most_common(100), columns=["skill", "count"])
    R.save_df(top_freq, "jobs_skills_top100.csv", index=False)

    # Co-occurrence for top 30
    labels, M = cooccurrence_matrix(job_skills_lists.tolist(), top=30)
    co_df = pd.DataFrame(M, index=labels, columns=labels)
    R.save_df(co_df.reset_index().rename(columns={"index": "skill"}),
              "jobs_skills_coocc_top30.csv", index=False)

    # Heatmap
    fig = plt.figure(figsize=(8, 7))
    ax = fig.add_subplot(111)
    im = ax.imshow(M, interpolation="nearest", aspect="auto", cmap="YlOrRd")
    ax.set_title("Job Skills Co-occurrence (top 30)")
    ax.set_xticks(np.arange(len(labels)))
    ax.set_yticks(np.arange(len(labels)))
    ax.set_xticklabels(labels, rotation=90, fontsize=7)
    ax.set_yticklabels(labels, fontsize=7)
    fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
    R.save_fig(fig, "jobs_skills_coocc_top30.png")
    plt.close()

    print("[OK] Saved jobs skills frequency & co-occurrence (top 100/30)")
else:
    print("[INFO] No explicit job skills column detected; skipping skills EDA.")
```

```
[OK] Saved jobs skills frequency & co-occurrence (top 100/30)
```

In [9]:

```
# ============================================================================
# CELL 8: Résumés - Basic Diagnostics
# ============================================================================
RES_TITLE_CANDS = ["current_job_title", "resume_title", "title", "headline"]
RES_SUM_CANDS = ["summary", "about", "description", "profile"]
RES_SKILL_CANDS = ["skills", "top_skills", "skillset", "core_skills", "resume_skills"]
RES_TARGET_CANDS = ["target_job_description", "target_role", "desired_role", "objective"]
```

```
res_title = col(resumes, RES_TITLE_CANDS, pd.Series([""] * len(resumes), index=resumes.index))
res_sum = col(resumes, RES_SUM_CANDS, pd.Series([""] * len(resumes), index=resumes.index))
res_skill = col(resumes, RES_SKILL_CANDS)
res_target = col(resumes, RES_TARGET_CANDS, pd.Series([""] * len(resumes), index=resumes.index))

# Target role counts
R.save_df(value_counts_df(res_target, name="target_role"),
          "resumes_target_role_counts.csv", index=False)

# Title length histogram
fig = plt.figure()
ax = fig.add_subplot(111)
ax.hist(res_title.fillna("").astype(str).str.len(), bins=40)
ax.set_title("Resume Title Length Distribution")
ax.set_xlabel("Characters")
ax.set_ylabel("Count")
R.save_fig(fig, "resumes_title_len_hist.png")
plt.close()

# Summary length histogram
sum_len = res_sum.fillna("").astype(str).str.len()
fig = plt.figure()
ax = fig.add_subplot(111)
ax.hist(np.clip(sum_len, 0, np.percentile(sum_len, 99)), bins=50)
ax.set_title("Resume Summary Length Distribution")
ax.set_xlabel("Characters (clipped at P99)")
ax.set_ylabel("Count")
R.save_fig(fig, "resumes_summary_len_hist.png")
plt.close()

print("[OK] Saved resume basic diagnostics")
```

```
[OK] Saved resume basic diagnostics
```

In [10]:

```
# ============================================================================
# CELL 9: Résumés - Skills Analysis (Top 100 Co-occurrence)
# ============================================================================
if res_skill is not None:
    res_skill_lists = parse_skills_column(res_skill)

    # Frequency
```

```
    freq = Counter([s for row in res_skill_lists for s in row])
    res_top = pd.DataFrame(freq.most_common(100), columns=["skill", "count"])
    R.save_df(res_top, "resumes_skills_top100.csv", index=False)

    # Co-occurrence for top 100 (as per your requirement)
    labels, M = cooccurrence_matrix(res_skill_lists.tolist(), top=100)
    co_df = pd.DataFrame(M, index=labels, columns=labels)
    R.save_df(co_df.reset_index().rename(columns={"index": "skill"}),
              "resumes_skills_coocc_top100.csv", index=False)

    # Heatmap
    fig = plt.figure(figsize=(12, 10))
    ax = fig.add_subplot(111)
    im = ax.imshow(M, interpolation="nearest", aspect="auto", cmap="YlOrRd")
    ax.set_title("Resume Skills Co-occurrence (top 100)")
    ax.set_xticks(np.arange(len(labels)))
    ax.set_yticks(np.arange(len(labels)))
    ax.set_xticklabels(labels, rotation=90, fontsize=5)
    ax.set_yticklabels(labels, fontsize=5)
    fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
    R.save_fig(fig, "resumes_skills_coocc_top100.png")
    plt.close()

    print("[OK] Saved resume skills frequency & co-occurrence (top 100)")
else:
    print("[INFO] No explicit resume skills column detected; skipping resume-skills EDA.")
```

```
[OK] Saved resume skills frequency & co-occurrence (top 100)
```

In [11]:

```
# ============================================================================
# CELL 10: Ontology - Family/Role/Skills Analysis
# ============================================================================
ONT_ROLE = ["job title", "role_title", "title", "role"]
ONT_FAM = ["family", "role_family", "job_family"]
ONT_SK = ["skills", "skills_core", "skills_required"]

role_col = col(ontology, ONT_ROLE, pd.Series([""] * len(ontology), index=ontology.index))
fam_col = col(ontology, ONT_FAM, pd.Series([""] * len(ontology), index=ontology.index))
sk_col = col(ontology, ONT_SK)

R.save_df(value_counts_df(fam_col, top=50, name="family"),
```

```
          "ontology_family_counts.csv", index=False)
R.save_df(value_counts_df(role_col, top=100, name="role_title"),
          "ontology_role_counts_top100.csv", index=False)

if sk_col is not None:
    ont_skill_lists = parse_skills_column(sk_col)
    freq = Counter([s for row in ont_skill_lists for s in row])
    top_ont = pd.DataFrame(freq.most_common(100), columns=["skill", "count"])
    R.save_df(top_ont, "ontology_skills_top100.csv", index=False)
    print("[OK] Saved ontology family/role counts + skills")
else:
    print("[INFO] No ontology skills column found")
```

```
[OK] Saved ontology family/role counts + skills
```

In [12]:

```
# ==========================================================================
# CELL 11: Résumé Distributions & Title Bigrams (FIXED)
# ==========================================================================
# Tokens per résumé (title + summary)
title_txt = res_title.fillna("").astype(str)
sum_txt = res_sum.fillna("").astype(str)
tokens_per_resume = tokenize_series(title_txt + " " + sum_txt).apply(len)

fig = plt.figure()
ax = fig.add_subplot(111)
ax.hist(tokens_per_resume, bins=40)
ax.set_title("Tokens per Résumé")
ax.set_xlabel("Token Count")
ax.set_ylabel("Résumés")
R.save_fig(fig, "resumes_tokens_per_resume_hist.png")
plt.close()

# Skills per résumé
if res_skill is not None:
    skills_per_resume = res_skill_lists.apply(len)
    fig = plt.figure()
    ax = fig.add_subplot(111)
    ax.hist(skills_per_resume, bins=40)
    ax.set_title("Skills per Résumé")
    ax.set_xlabel("Skill Count")
    ax.set_ylabel("Résumés")
```

```
    R.save_fig(fig, "resumes_skills_per_resume_hist.png")
    plt.close()

# Title entropy (Shannon entropy over characters)
def shannon_entropy(s):
    s = "".join(sorted(s.lower()))
    if not s:
        return 0.0
    counts = Counter(s)
    total = len(s)
    import math
    return -sum((c / total) * math.log2(c / total) for c in counts.values())

title_entropy = title_txt.apply(shannon_entropy)
fig = plt.figure()
ax = fig.add_subplot(111)
ax.hist(title_entropy, bins=40)
ax.set_title("Title Entropy Distribution")
ax.set_xlabel("Entropy (bits)")
ax.set_ylabel("Résumés")
R.save_fig(fig, "resumes_title_entropy_hist.png")
plt.close()

# 🔧 FIXED: Bigrams in JOB TITLES (not names)
def bigrams(tokens):
    return [f"{tokens[i]} {tokens[i+1]}" for i in range(len(tokens) - 1)]

# Use current_job_title column, NOT name
job_title_col = col(resumes, ["current_job_title", "title", "job_title"])
if job_title_col is not None:
    title_bigrams = Counter()
    for toks in tokenize_series(job_title_col):
        if len(toks) >= 2:  # Need at least 2 tokens for bigrams
            title_bigrams.update(bigrams(toks))

    if len(title_bigrams) > 0:
        top30_bi = pd.DataFrame(title_bigrams.most_common(30), columns=["bigram", "count"])
        R.save_df(top30_bi, "resumes_title_bigrams_top30.csv", index=False)

        # Visualization
        fig = plt.figure(figsize=(8, 6))
```

```
        ax = fig.add_subplot(111)
        ax.barh(top30_bi["bigram"][::-1], top30_bi["count"][::-1])
        ax.set_title("Job Title Bigrams (top 30)")
        ax.set_xlabel("Frequency")
        R.save_fig(fig, "resumes_title_bigrams_top30.png")
        plt.close()
        print("[OK] Saved title bigrams (FIXED - using job titles)")
    else:
        print("[WARN] No bigrams found in job titles")
else:
    print("[WARN] No current_job_title column found for bigram extraction")
```

```
[OK] Saved title bigrams (FIXED - using job titles)
```

In [13]:

```
# ============================================================================
# CELL 12: Job Long-Tail & Soft:Hard Skills Ratio (FINAL FIX - Quote Handling)
# ============================================================================
if skills_col is not None:
    job_skills_lists = parse_skills_column(skills_col)
    freq = Counter([s for row in job_skills_lists for s in row])

    # Long-tail plot
    counts = np.array(sorted(freq.values(), reverse=True))
    fig = plt.figure()
    ax = fig.add_subplot(111)
    ax.loglog(np.arange(1, len(counts) + 1), counts)
    ax.set_title("Job Skills Long-Tail Distribution (log-log)")
    ax.set_xlabel("Rank")
    ax.set_ylabel("Frequency")
    ax.grid(True, alpha=0.3)
    R.save_fig(fig, "jobs_skills_longtail_loglog.png")
    plt.close()

    # Comprehensive soft skills dictionary
    SOFT_RAW = """
    communication communications communicate communicating
    problem-solving problem solving problems problem
    teamwork team work team-work teaming collaboration collaborate collaborating
    leadership leader leading lead manage management managing manager
    interpersonal skills interpersonal people skills
    time-management time management managing time
```

```
    organizational skills organizational organization organizing organise
    attention to detail attention detail oriented meticulous
    adaptability adaptable adapt flexible flexibility
    creativity creative innovate innovation innovative
    critical thinking critical think analytical thinking analytical analysis
    presentation skills presentation presenting present
    writing written write documentation document
    relationship building relationship networking network
    negotiation negotiate negotiating
    conflict resolution conflict resolve resolving
    coaching mentor mentoring mentorship
    stakeholder management stakeholder
    employee relations employee
    customer service customer client service clients
    strategic thinking strategic strategy planning strategic planning
    self-motivation self-motivated motivated motivation motivate
    initiative proactive take initiative
    troubleshooting troubleshoot debugging debug
    decision making decision decisions
    active listening listening listen
    emotional intelligence emotional empathy empathetic
    work ethic ethics ethical
    professionalism professional
    accountability accountable responsible responsibility
    reliability reliable dependable
    punctuality punctual
    multitasking multi-tasking juggle prioritize prioritizing
    stress management stress handle pressure
    team collaboration team player
    performance management performance
    project management project
    change management change
    delegation delegate
    feedback giving feedback receiving
    persuasion persuade persuasive
    public speaking speaking speech
    storytelling story
    active learning learning eager learn
    teaching teach training train
    facilitation facilitate
    mediation mediate
```

```
    consensus building consensus
    influence influencing
    sales selling sell
    customer-focused customer focused
    client-focused client focused
    results-driven results driven goal-oriented goal oriented
    detail-oriented detail oriented
    quality-focused quality focus
    continuous improvement improvement
    cross-functional cross functional
    stakeholder engagement engagement
    microsoft office suite microsoft office ms office
    budgeting scheduling coordination administration
    research reporting meeting interviewing
    """

    # Create SOFT set
    SOFT = set()
    for term in SOFT_RAW.split():
        cleaned = term.strip().lower()
        if cleaned:
            SOFT.add(cleaned)

    #  🔧 KEY FIX: Helper function to strip quotes from skill names
    def strip_quotes(skill):
        """Remove leading/trailing single or double quotes."""
        skill = skill.strip()
        if (skill.startswith("'") and skill.endswith("'")) or \
           (skill.startswith('"') and skill.endswith('")):
            return skill[1:-1]
        return skill

    # Reclassify with quote stripping
    SOFT_NORMALIZED = {strip_quotes(s) for s in SOFT}

    # Count per job (strip quotes when checking)
    soft_counts = []
    hard_counts = []
    for row in job_skills_lists:
        s = sum(1 for x in row if strip_quotes(x) in SOFT_NORMALIZED)
        h = sum(1 for x in row if strip_quotes(x) not in SOFT_NORMALIZED)
```

```
        soft_counts.append(s)
        hard_counts.append(h)

    sh_df = pd.DataFrame({"soft": soft_counts, "hard": hard_counts})
    R.save_df(sh_df.describe().T.reset_index().rename(columns={"index": "metric"}),
              "jobs_soft_hard_stats.csv", index=False)

    # Classify ALL skills for reporting
    soft_list = []
    hard_list = []
    for skill, count in freq.items():
        if strip_quotes(skill) in SOFT_NORMALIZED:
            soft_list.append((skill, count))
        else:
            hard_list.append((skill, count))

    soft_list.sort(key=lambda x: x[1], reverse=True)
    hard_list.sort(key=lambda x: x[1], reverse=True)

    # Save classifications
    classifications = []
    for skill, count in soft_list[:50]:
        classifications.append({'skill': skill, 'type': 'soft', 'count': count})
    for skill, count in hard_list[:50]:
        classifications.append({'skill': skill, 'type': 'hard', 'count': count})
    R.save_df(pd.DataFrame(classifications), "jobs_skill_classifications_top50.csv", index=False)

    # Stats
    total_soft = sum(soft_counts)
    total_hard = sum(hard_counts)
    total_skills = total_soft + total_hard

    print(f"\n{'='*60}")
    print("SOFT vs HARD SKILLS ANALYSIS")
    print(f"{'='*60}")
    print(f"Soft skills in dictionary: {len(SOFT_NORMALIZED):,}")
    print(f"Soft skills found in data: {len(soft_list):,}")
    print(f"Hard skills found in data: {len(hard_list):,}")
    print(f"\nTotal skill mentions:")
    print(f"  Soft:  {total_soft:,} ({100*total_soft/total_skills:.1f}%)")
    print(f"  Hard:  {total_hard:,} ({100*total_hard/total_skills:.1f}%)")
```

```
    print(f"\nPer job (mean):")
    print(f"  Soft:  {np.mean(soft_counts):.2f} ± {np.std(soft_counts):.2f}")
    print(f"  Hard:  {np.mean(hard_counts):.2f} ± {np.std(hard_counts):.2f}")
    print(f"\nJobs with ≥1 soft skill: {sum(1 for s in soft_counts if s > 0)}/{len(soft_counts)} ({100*sum(1 for s in soft_cou
    print(f"Jobs with ≥1 hard skill: {sum(1 for s in hard_counts if s > 0)}/{len(hard_counts)} ({100*sum(1 for s in hard_count
    print(f"{'='*60}\n")

    # Top 10 by category
    print("Top 10 Soft Skills:")
    for i, (skill, count) in enumerate(soft_list[:10], 1):
        print(f"  {i:2d}. {skill:35s} → {count:3d} jobs ({100*count/len(jobs):.1f}%)")

    print("\nTop 10 Hard Skills:")
    for i, (skill, count) in enumerate(hard_list[:10], 1):
        print(f"  {i:2d}. {skill:35s} → {count:3d} jobs ({100*count/len(jobs):.1f}%)")

    # Ratio histogram
    ratios = np.array(soft_counts) / (np.array(hard_counts) + 0.1)
    fig = plt.figure()
    ax = fig.add_subplot(111)
    ax.hist(ratios, bins=40, edgecolor='black', alpha=0.7)
    ax.set_title("Soft:Hard Skill Ratio per Job")
    ax.set_xlabel("Ratio (soft/hard)")
    ax.set_ylabel("Jobs")
    ax.axvline(np.mean(ratios), color='red', linestyle='--', linewidth=2,
               label=f'Mean: {np.mean(ratios):.2f}')
    ax.legend()
    R.save_fig(fig, "jobs_soft_hard_ratio_hist.png")
    plt.close()

    print("\n[OK] Soft vs Hard skills analysis complete!")
```

```
============================================================
SOFT vs HARD SKILLS ANALYSIS
============================================================
Soft skills in dictionary: 194
Soft skills found in data: 55
Hard skills found in data: 3,804

Total skill mentions:
  Soft:  2,241 (19.5%)
  Hard:  9,249 (80.5%)

Per job (mean):
  Soft:  3.73 ± 1.58
  Hard:  15.41 ± 4.97

Jobs with ≥1 soft skill: 594/600 (99.0%)
Jobs with ≥1 hard skill: 600/600 (100.0%)
============================================================

Top 10 Soft Skills:
   1. 'communication'                    → 541 jobs (90.2%)
   2. 'adaptability'                     → 297 jobs (49.5%)
   3. 'teamwork'                         → 286 jobs (47.7%)
   4. 'collaboration'                    → 196 jobs (32.7%)
   5. 'leadership'                       → 168 jobs (28.0%)
   6. 'negotiation'                      →  58 jobs (9.7%)
   7. 'self-motivation'                  →  48 jobs (8.0%)
   8. 'troubleshooting'                  →  41 jobs (6.8%)
   9. 'initiative'                       →  38 jobs (6.3%)
  10. 'networking'                       →  36 jobs (6.0%)

Top 10 Hard Skills:
   1. 'problem solving'                  → 466 jobs (77.7%)
   2. 'interpersonal skills'             → 157 jobs (26.2%)
   3. 'time management'                  → 140 jobs (23.3%)
   4. 'organizational skills'            → 122 jobs (20.3%)
   5. 'attention to detail'              → 122 jobs (20.3%)
   6. 'project management'               → 122 jobs (20.3%)
   7. 'relationship building'            → 121 jobs (20.2%)
   8. 'data analysis'                    → 118 jobs (19.7%)
   9. 'customer service'                 → 107 jobs (17.8%)
```

```
 10. 'analytical skills'                 →  62 jobs (10.3%)

[OK] Soft vs Hard skills analysis complete!
```

In [14]:

```python
# ============================================================================
# CELL 13: Cross-Dataset Vocabulary Overlap (FIXED)
# ============================================================================
def ensure_series(maybe_series, length, index=None):
    """Helper to ensure we always pass a Series to parse_skills_column."""
    if maybe_series is not None:
        return maybe_series
    if index is None:
        return pd.Series([""] * length)
    return pd.Series([""] * length, index=index)

# Build vocabularies with explicit checks
res_sk_series = ensure_series(res_skill, length=len(resumes), index=resumes.index)
job_sk_series = ensure_series(skills_col, length=len(jobs), index=jobs.index)
ont_sk_series = ensure_series(sk_col, length=len(ontology), index=ontology.index)

res_vocab = set(s for row in parse_skills_column(res_sk_series) for s in row)
job_vocab = set(s for row in parse_skills_column(job_sk_series) for s in row)
ont_vocab = set(s for row in parse_skills_column(ont_sk_series) for s in row)

def jaccard(a, b):
    if not a and not b:
        return 1.0
    if not a or not b:
        return 0.0
    return len(a & b) / len(a | b)

jac_rj = jaccard(res_vocab, job_vocab)
jac_ro = jaccard(res_vocab, ont_vocab)
jac_jo = jaccard(job_vocab, ont_vocab)

# Save metrics
metrics = {
    "jaccard_resume_job": float(jac_rj),
    "jaccard_resume_ontology": float(jac_ro),
    "jaccard_job_ontology": float(jac_jo),
    "vocab_sizes": {
```

```
        "resumes": len(res_vocab),
        "jobs": len(job_vocab),
        "ontology": len(ont_vocab),
        "total_unique": len(res_vocab | job_vocab | ont_vocab)
    }
}
R.save_json(metrics, "vocab_jaccard.json")

print(f"\n{'='*60}")
print("VOCABULARY OVERLAP ANALYSIS")
print(f"{'='*60}")
print(f"Résumé vocab size:  {len(res_vocab):,}")
print(f"Job vocab size:     {len(job_vocab):,}")
print(f"Ontology vocab size: {len(ont_vocab):,}")
print(f"Total unique skills: {len(res_vocab | job_vocab | ont_vocab):,}")
print(f"\nJaccard Overlaps:")
print(f"  Résumé ↔ Job:      {jac_rj:.4f} ({jac_rj*100:.2f}%)")
print(f"  Résumé ↔ Ontology: {jac_ro:.4f} ({jac_ro*100:.2f}%)")
print(f"  Job ↔ Ontology:    {jac_jo:.4f} ({jac_jo*100:.2f}%)")
print(f"{'='*60}\n")

# Venn diagram data (fallback if matplotlib-venn not available)
venn_counts = pd.DataFrame({
    "set": ["resume", "job", "ontology", "resumenjob", "resumenontology",
            "jobnontology", "all3"],
    "count": [
        len(res_vocab),
        len(job_vocab),
        len(ont_vocab),
        len(res_vocab & job_vocab),
        len(res_vocab & ont_vocab),
        len(job_vocab & ont_vocab),
        len(res_vocab & job_vocab & ont_vocab)
    ]
})
R.save_df(venn_counts, "vocab_venn_counts.csv", index=False)

# Try to create Venn diagram
try:
    from matplotlib_venn import venn3
    fig = plt.figure(figsize=(10, 8))
```

```
    ax = fig.add_subplot(111)
    venn3([res_vocab, job_vocab, ont_vocab],
          set_labels=("Résumé", "Job", "Ontology"))
    ax.set_title("Skill Vocabulary Overlap Across Datasets")
    R.save_fig(fig, "vocab_venn_rjo.png")
    plt.close()
    print("[OK] Venn diagram created")
except ImportError:
    print("[INFO] matplotlib-venn not installed; saved counts CSV instead")
```

```
============================================================
VOCABULARY OVERLAP ANALYSIS
============================================================
Résumé vocab size:  72
Job vocab size:     3,859
Ontology vocab size: 699
Total unique skills: 4,574

Jaccard Overlaps:
  Résumé ↔ Job:      0.0005 (0.05%)
  Résumé ↔ Ontology: 0.0738 (7.38%)
  Job ↔ Ontology:    0.0004 (0.04%)
============================================================

[OK] Venn diagram created
```

In [15]:

```
# ============================================================================
# CELL 14: Missingness Matrix
# ============================================================================
key_cols = []
for name_set in [
    ["job_title", "title"],
    ["job_description", "description"],
    ["job_skill_set", "skills"],
    ["current_job_title", "resume_title", "title"],
    ["summary", "about"],
    ["skills", "resume_skills"],
    ["job title", "role_title", "title"],
    ["family"]
]:
    for n in name_set:
```

```
        if n in jobs.columns or n in resumes.columns or n in ontology.columns:
            key_cols.append(n)
            break

key_df = pd.DataFrame(index=range(max(len(jobs), len(resumes), len(ontology))))

for c in key_cols:
    if c in jobs.columns:
        key_df[f"jobs:{c}"] = jobs[c].reindex(key_df.index).isna()
    elif c in resumes.columns:
        key_df[f"res:{c}"] = resumes[c].reindex(key_df.index).isna()
    elif c in ontology.columns:
        key_df[f"ont:{c}"] = ontology[c].reindex(key_df.index).isna()

if len(key_df.columns) > 0:
    fig = plt.figure(figsize=(max(8, len(key_df.columns) * 0.5), 5))
    ax = fig.add_subplot(111)
    ax.imshow(~key_df.values, aspect="auto", interpolation="nearest", cmap="RdYlGn")
    ax.set_yticks([])
    ax.set_xticks(np.arange(len(key_df.columns)))
    ax.set_xticklabels(key_df.columns, rotation=90, fontsize=7)
    ax.set_title("Data Completeness Matrix (green=present, red=missing)")
    R.save_fig(fig, "missingness_matrix.png")
    plt.close()
    print("[OK] Saved missingness matrix")
```

```
[OK] Saved missingness matrix
```

In [16]:

```
# ============================================================================
# CELL 15: Outlier Detection - Résumés with Unusual Skill Counts
# ============================================================================
if res_skill is not None and 'skills_per_resume' in locals():
    low_thr = float(np.percentile(skills_per_resume, 5))
    high_thr = float(np.percentile(skills_per_resume, 95))

    out_low = resumes.loc[skills_per_resume <= low_thr]
    out_high = resumes.loc[skills_per_resume >= high_thr]

    R.save_df(pd.DataFrame({"low_threshold": [low_thr], "high_threshold": [high_thr]}),
              "resume_skill_outlier_thresholds.csv", index=False)
    R.save_df(out_low.head(50), "resume_skill_outliers_low_sample.csv", index=False)
```

```
    R.save_df(out_high.head(50), "resume_skill_outliers_high_sample.csv", index=False)

    print(f"[OK] Outlier thresholds: Low ≤ {low_thr:.1f}, High ≥ {high_thr:.1f}")
    print(f"     Low outliers: {len(out_low)}, High outliers: {len(out_high)}")
```

```
[OK] Outlier thresholds: Low ≤ 4.0, High ≥ 8.0
     Low outliers: 223, High outliers: 267
```

In [17]:

```
# ============================================================================
# CELL 16: Ontology Bipartite Preview (Role → Skill)
# ============================================================================
if role_col is not None and sk_col is not None:
    lists = parse_skills_column(sk_col)

    # Top 40 skills for readability
    freq = Counter([s for row in lists for s in row])
    top_sk = [s for s, _ in freq.most_common(40)]

    # Top 40 roles
    uniq_roles = role_col.fillna("").astype(str).str.lower().unique()[:40]

    Rmap = {r: i for i, r in enumerate(uniq_roles)}
    Smap = {s: i for i, s in enumerate(top_sk)}
    M = np.zeros((len(uniq_roles), len(top_sk)), dtype=int)

    for r, items in zip(role_col.fillna("").astype(str).str.lower(), lists):
        if r in Rmap:
            for s in set(items):
                if s in Smap:
                    M[Rmap[r], Smap[s]] += 1

    fig = plt.figure(figsize=(12, 10))
    ax = fig.add_subplot(111)
    im = ax.imshow(M, aspect="auto", interpolation="nearest", cmap="YlOrRd")
    ax.set_yticks(np.arange(len(uniq_roles)))
    ax.set_yticklabels(uniq_roles, fontsize=6)
    ax.set_xticks(np.arange(len(top_sk)))
    ax.set_xticklabels(top_sk, rotation=90, fontsize=6)
    ax.set_title("Ontology: Role → Skill Bipartite Matrix")
    fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
    R.save_fig(fig, "ontology_role_skill_bipartite.png")
```

```
    plt.close()
    print("[OK] Saved ontology bipartite matrix")
```

```
[OK] Saved ontology bipartite matrix
```

In [18]:

```
# ============================================================================
# CELL 17: Certifications per Family (if available)
# ============================================================================
cert_col = col(ontology, ["certifications", "certs", "top_certs"])
if fam_col is not None and cert_col is not None:
    cert_lists = parse_skills_column(cert_col)
    df_c = pd.DataFrame({
        "family": fam_col.fillna("").astype(str).str.lower(),
        "certs": cert_lists
    })

    rows = []
    for fam, group in df_c.groupby("family"):
        pool = Counter([c for row in group["certs"] for c in row])
        for cert, cnt in pool.most_common(10):
            rows.append((fam, cert, cnt))

    if rows:
        top_certs = pd.DataFrame(rows, columns=["family", "certification", "count"])
        R.save_df(top_certs, "ontology_top_certs_per_family.csv", index=False)
        print("[OK] Saved top certifications per family")
```

```
[OK] Saved top certifications per family
```

In [19]:

```
# ============================================================================
# CELL 18: Insights-to-Actions Report (Markdown)
# ============================================================================
lines = []
lines.append("# Phase 02 – Insights → Actions")
lines.append("")
lines.append(f"- Jobs loaded: {len(jobs):,}; Résumés: {len(resumes):,}; Ontology rows: {len(ontology):,}")

if 'title_entropy' in locals():
    lines.append(f"- Title entropy median: {float(title_entropy.median()):.2f} (suggests long-tail → normalize casing/stopword

if 'skills_per_resume' in locals():
```

```
    lines.append(f"- Résumé skills per record: median={float(np.median(skills_per_resume)):.1f}, P95={float(np.percentile(skil

lines.append(f"- Skill vocab Jaccard overlap – Résumé↔Job={jac_rj:.4f}, Résumé↔Ont={jac_ro:.4f}, Job↔Ont={jac_jo:.4f}")
lines.append("  **CRITICAL**: Near-zero overlap confirms vocabulary mismatch → requires aggressive normalization")
lines.append("")
lines.append("## Key Findings:")
lines.append(f"- Total unique skills: {len(res_vocab | job_vocab | ont_vocab):,}")
lines.append(f"- Only {len(res_vocab & job_vocab & ont_vocab)} skill(s) appear in all three datasets")
lines.append("- Co-occurrence heatmaps reveal tight clusters → use as cues for alias groups")
lines.append("- Long-tail plots confirm heavy-tail skills → cap rare skills or map to parents")
lines.append("")
lines.append("## Issues Identified:")
lines.append("- **Title bigram bug** (now fixed): Was extracting from names instead of job titles")
lines.append(f"- **Duplicate résumé IDs**: Found {1200 - resumes['resume_id'].nunique()} duplicates to investigate")
lines.append("- **High missingness**: 36.75% current titles, 76.17% previous titles (expected for fresh graduates)")
lines.append("")
lines.append("## Next Actions:")
lines.append("1. Skill normalization (aliases/stemming) - Phase 03")
lines.append("2. Soft/hard skill tagging - refine classification logic")
lines.append("3. Review and handle outliers")
lines.append("4. Drop or impute empty fields")
lines.append("5. Verify duplicate résumé IDs and ontology roles")

md = "\n".join(lines)
rep_dir = P.phase_dir(2)["tab"].parent / "reports"
rep_dir.mkdir(parents=True, exist_ok=True)
out_path = rep_dir / "insights_to_actions.md"
out_path.write_text(md, encoding="utf-8")
print(f"\n[OK] Saved insights report: {out_path}")
```

```
[OK] Saved insights report: C:\Users\naikt\OneDrive\Desktop\tanmay_dissertation\outputs\phase02\reports\insights_to_actions.md
```

In [20]:

```
# ============================================================================
# CELL 19: Summary JSON & Integrity Checks (FIXED - Direct File Write)
# ============================================================================
summary = {
    "phase": "02_eda",
    "datasets": {
        "jobs_rows": int(len(jobs)),
        "resumes_rows": int(len(resumes)),
        "ontology_rows": int(len(ontology))
```

```
    },
    "data_quality": {
        "jobs_title_nulls": int(col(jobs, TITLE_CANDS).isna().sum() if col(jobs, TITLE_CANDS) is not None else len(jobs)),
        "jobs_desc_nulls": int(col(jobs, DESC_CANDS).isna().sum() if col(jobs, DESC_CANDS) is not None else len(jobs)),
        "resume_current_title_nulls": int(res_title.isna().sum()),
        "resume_skill_nulls": int(res_sk_series.isna().sum())
    },
    "vocabulary": {
        "resume_unique_skills": len(res_vocab),
        "job_unique_skills": len(job_vocab),
        "ontology_unique_skills": len(ont_vocab),
        "total_unique": len(res_vocab | job_vocab | ont_vocab),
        "jaccard_resume_job": float(jac_rj),
        "jaccard_resume_ontology": float(jac_ro),
        "jaccard_job_ontology": float(jac_jo)
    }
}

if 'skills_per_resume' in locals():
    summary["skills_stats"] = {
        "resume_median_skills": float(np.median(skills_per_resume)),
        "resume_mean_skills": float(np.mean(skills_per_resume)),
        "resume_p95_skills": float(np.percentile(skills_per_resume, 95))
    }

# 🔧 FIX: Write directly using Python's json module (more reliable)
tab = P.phase_dir(2)["tab"]
summary_path = tab / "p2_summary.json"
with open(summary_path, 'w', encoding='utf-8') as f:
    json.dump(summary, f, indent=2)

# Verify it was created
if summary_path.exists():
    print(f"✅ Summary saved: {summary_path}")
else:
    print(f"❌ Failed to save: {summary_path}")

R.stamp(f"Phase 02 summary: {json.dumps(summary, indent=2)}")

print("\n" + "="*60)
print("PHASE 02 SUMMARY")
```

```
print("="*60)
print(json.dumps(summary, indent=2))
print("="*60)
```

```
✅ Summary saved: C:\Users\naikt\OneDrive\Desktop\tanmay_dissertation\outputs\phase02\tables\p2_summary.json

============================================================
PHASE 02 SUMMARY
============================================================
{
  "phase": "02_eda",
  "datasets": {
    "jobs_rows": 600,
    "resumes_rows": 1200,
    "ontology_rows": 493
  },
  "data_quality": {
    "jobs_title_nulls": 0,
    "jobs_desc_nulls": 0,
    "resume_current_title_nulls": 441,
    "resume_skill_nulls": 0
  },
  "vocabulary": {
    "resume_unique_skills": 72,
    "job_unique_skills": 3859,
    "ontology_unique_skills": 699,
    "total_unique": 4574,
    "jaccard_resume_job": 0.0005090353779587681,
    "jaccard_resume_ontology": 0.07381615598885793,
    "jaccard_job_ontology": 0.0004389815627743635
  },
  "skills_stats": {
    "resume_median_skills": 6.0,
    "resume_mean_skills": 6.1025,
    "resume_p95_skills": 8.0
  }
}
============================================================
```

In [21]:

```
# ============================================================================
# CELL 20: Final Validation & Completion
```

```python
# ============================================================================
print("\n" + "="*60)
print("DELIVERABLES CHECK")
print("="*60)

tab = P.phase_dir(2)["tab"]
fig = P.phase_dir(2)["fig"]

must_tables = [
    "p2_jobs_profile.csv", "p2_resumes_profile.csv", "p2_ontology_profile.csv",
    "jobs_skills_top100.csv", "resumes_skills_top100.csv",
    "resumes_skills_coocc_top100.csv",
    "ontology_role_counts_top100.csv", "ontology_family_counts.csv",
    "ontology_skills_top100.csv", "jobs_soft_hard_stats.csv",
    "vocab_jaccard.json", "p2_summary.json"
]

missing = [f for f in must_tables if not (tab / f).exists()]
if missing:
    print(f"⚠️  Missing tables: {missing}")
else:
    print("✅ All required tables present")

figs = list(fig.glob("*.png"))
print(f"✅ Generated {len(figs)} figures")

rep = P.phase_dir(2)["tab"].parent / "reports"
if (rep / "insights_to_actions.md").exists():
    print("✅ Insights report created")
else:
    print("⚠️  Insights report missing")

print("="*60)

R.stamp("Phase 02 complete")
print("\n✅ Phase 02 EDA complete. Tables & figures saved under outputs/phase02/")
print("\n🔧 FIXES APPLIED:")
print("   1. Title bigrams now extracted from job titles (not names)")
print("   2. Soft skills classification improved")
print("   3. Vocabulary overlap properly calculated")
```

```
print("   4. Better error handling for missing columns")
print("   5. Comprehensive summary with all metrics")
```

```
============================================================
DELIVERABLES CHECK
============================================================
✅ All required tables present
✅ Generated 16 figures
✅ Insights report created
============================================================

✅ Phase 02 EDA complete. Tables & figures saved under outputs/phase02/

🔧 FIXES APPLIED:
   1. Title bigrams now extracted from job titles (not names)
   2. Soft skills classification improved
   3. Vocabulary overlap properly calculated
   4. Better error handling for missing columns
   5. Comprehensive summary with all metrics
```